

SAARA KETHA HARVEST HUB · ISE_WE_0101_23

Epic 2 – Produce Collection, Weighing & Quality Management

Concepts, validations, demo script and viva questions

1. Epic overview
2. Core concepts
3. Validations
4. Suggested live demo
5. Likely viva questions and model answers
6. Know your limitations
7. Where to find it in the code
8. Appendix – system foundations (same in every guide)

Study guide generated from the implemented system · October 2026

1. Epic overview

Epic 2 covers the **physical journey of produce into the centre**: the farmer books a delivery, the officer registers the delivery and weighs it, the inspector grades it and records accepted / rejected quantities, the system generates a **batch code** for accepted produce and a numbered **collection receipt** for the farmer.

Stakeholders: Farmer, Collection Centre Officer, Quality Inspector.

1.1 Story map

Story	What happens	Screen	API	Tables / DB function
E2-US1 Sprint 1	Farmer schedules a delivery appointment	ScheduleDelivery , MyAppointments	POST /api/appointments , PATCH /:id/status	delivery_appointments
E2-US2 Sprint 1	Officer registers scheduled or walk-in delivery	RegisterCollection , DeliveryAppointments	POST /api/collections	produce_collections
E2-US3 Sprint 1	Gross + container weight → net weight	WeighProduce	POST /api/collections/:id/weigh	produce_collections (net weight is a generated column)
E2-US4 Sprint 1	Inspector grades the produce	PendingInspections , CreateInspection	POST /api/inspections	finalize_inspection() → quality_inspections
E2-US5 Sprint 1	Accepted / rejected quantities with reasons	CreateInspection	same call	collection_rejections
E2-US6	Evidence photos	CreateInspection , InspectionHistory	POST/GET/DELETE /api/inspections/:id/images	inspection_evidence_images + storage bucket evidence-images
E2-US7	Batch number + code for traceability	(automatic) shown in CollectionHistory	inside finalize_inspection()	inventory_batches, inventory_transactions
E2-US8	Numbered collection receipt, status tracking, farmer confirms	CollectionReceiptCard , CollectionDetail , MyCollections	POST /api/collections/:id/receipt , /receipt/confirm	collection_receipts

Story	What happens	Screen	API	Tables / DB function
E2-US9	Prevent incorrect weights / quantities / approvals	all of the above	API rules + database constraints + RPC	CHECK constraints, <code>finalize_inspection()</code>

2. Core concepts

2.1 The collection life-cycle (state machine)

```

Appointment:  scheduled → confirmed → arrived → completed      (also: cancelled, no_show)
Collection:   scheduled → arrived → weighed → pending_inspection → under_inspection
              → accepted | partially_accepted | rejected
              → added_to_inventory → payment_pending → completed

```

A **state machine** means each status can only move to certain next statuses (a transition table in the controller). Trying anything else returns **409 Conflict**, for example "Collection cannot move from *scheduled* to *accepted*". This prevents skipping weighing or inspection.

2.2 Appointments vs walk-ins

- A **scheduled delivery** links the collection to an appointment (`appointment_id`); the appointment becomes **completed** automatically when the inspection is finalised.
- A **walk-in** is a collection with no appointment – the officer registers it directly. Both end up as the same kind of collection record, so everything after (weighing, inspection, payment) is identical.
- **Capacity**: each centre accepts a maximum number of appointments per day (system setting, default 60), so queues stay manageable.

2.3 Net weight is calculated, never typed

`net_weight_kg = gross_weight_kg - container_weight_kg` is a PostgreSQL **generated column** (`GENERATED ALWAYS AS ... STORED`). Nobody – not the API, not the UI – can write a wrong net weight; the database computes it from the two inputs. This is a strong answer for "how do you avoid manual calculation errors?".

2.4 One transaction for inspection – `finalize_inspection()`

When the inspector submits, a single PostgreSQL function does **everything atomically** (all or nothing):

1. Validates the collection state and the quantities (see section 3.4).
2. Inserts the `quality_inspections` row and one `collection_rejections` row per rejection reason.
3. Sets the collection status: **rejected** (grade rejected), **partially_accepted** (some rejected), or **accepted**.
4. Marks the linked appointment **completed**.
5. If accepted quantity > 0: generates a **batch number** (`generate_batch_no()`), looks up the current price, computes the **expiry date** from the crop's shelf-life days, creates the `inventory_batches` row (QR value `HH-<batch_no>`) and a "received" ledger entry in `inventory_transactions` .
6. Creates the numbered **collection receipt** (`RCT-YYYY-000001` style via `next_receipt_no()`) and notifies the farmer.

Why a database function? If the server crashed half-way in plain code you could have an inspection without a batch or a batch without a receipt. In one transaction either everything is saved or nothing is.

2.5 Traceability (E2-US7)

The batch stores `collection_id`, `farmer_id`, crop, grade and expiry. Later, orders allocate stock from batches, so any sold kilo can be traced back to the farmer and the delivery day: **farmer** → **collection** → **inspection** → **batch** → **order** → **invoice**.

2.6 Receipts and farmer confirmation (E2-US8)

- Receipt number format `RCT-<year>-<6-digit sequence>`, unique per collection (one receipt per collection).
- The farmer sees the receipt, can **print / save as PDF**, and either **confirm** it or **dispute** it (a note explaining the problem is mandatory when disputing). The officer is notified of disputes.
- Receipt status: `issued` → `confirmed` | `disputed`.

3. Validations

3.1 Appointment – E2-US1

Rule	Result
Farmer must be verified and account not suspended	403 "...registration has not been verified yet. Verification is required before using collection services."
centre, crop category and date are required	400
Estimated quantity > 0 and ≤ 100,000 kg	400 "must be greater than zero" / "looks too large"
Farmers: at least 1 day in advance ; staff: not in the past	400 "Deliveries must be scheduled at least one day in advance"
At most 90 days ahead	400
Centre must exist and be active	400
No duplicate for the same farmer + crop + centre + date	409 "You already have an appointment for this crop, centre and date"
Daily capacity of the centre	409 "This centre is fully booked on that date. Please choose another day."
Status change must follow the transition table	409 "Appointment is completed and cannot be changed to ..."
Farmers can only cancel , and only their own	403

Reference numbers are generated by the server; a unique-violation retry loop (error code 23505) guarantees no two appointments share a number even under simultaneous requests.

3.2 Registering the delivery – E2-US2

Rule	Result
Only the collection centre officer may register collections	403 for other roles

Rule	Result
Zod collectionSchema : farmer_id, centre_id, category_id are valid UUIDs; variety optional	400 with field errors
If an appointment is given: it must exist, belong to the same farmer , and be scheduled / confirmed	404 / 409
One collection per appointment	409 "A collection has already been registered for this appointment"
Farmer must be verified (same check as booking)	403

3.3 Weighing – E2-US3

Rule	Result
Gross and container are numbers; gross > 0 and container ≥ 0	400 "Gross and container weights must be valid numbers..."
Gross must be greater than container (a net weight of zero or negative is impossible)	400 "Gross weight must be greater than container weight"; also a database CHECK ck_net_weight_positive
Weights can only be changed while the collection is scheduled / arrived / weighed / pending inspection	409 "Weights cannot be changed once a collection is ..."
Net weight	generated by the database , status moves to pending_inspection
Only the officer may weigh	403 for others

3.4 Inspection – E2-US4 / US5 / US9 (inside finalize_inspection())

Rule	Error code → HTTP
Collection must exist	NOT_FOUND → 404
Net weight must already be recorded (cannot inspect before weighing)	INVALID_STATE → 409
Collection status must allow inspection	INVALID_STATE → 409
Only one inspection per collection	INVALID_STATE → 409 "an inspection already exists"
Quantities are zero or greater	INVALID_QTY → 400
accepted + rejected must equal the net weight	INVALID_QTY → 400 "accepted (x kg) + rejected (y kg) must equal the net weight (z kg)"
A rejected grade cannot have accepted quantity	INVALID_QTY → 400
Grades A / B / C require accepted quantity > 0	INVALID_QTY → 400
If anything is rejected, at least one rejection reason is required	INVALID_QTY → 400
Rejection line quantities cannot exceed total rejected	INVALID_QTY → 400
Grade must be grade_a, grade_b, grade_c or rejected; quantities must be numbers	400 (API, before the database)

The function raises messages like `INVALID_QTY: ...; utils/rpcError.ts` maps the code to the right HTTP status (`NOT_FOUND→404`, `INVALID_QTY→400`, `INVALID_STATE→409`). The same checks exist in the inspection form so the inspector sees the message immediately.

3.5 Evidence images – E2-US6

- Images are sent as base64 data URIs and stored in the `evidence-images` bucket; each gets a record with caption and type (general, defect, label, packaging, rejected area).
- 404 if the inspection does not exist, 400 if no images were sent; only staff roles (inspector, officer, admin) may upload or delete.
- The frontend page previously called an API method that did not exist, so uploads silently failed – this was fixed by adding it.

3.6 Receipts – E2-US8

Rule	Result
Receipt is created only after the inspection exists	409 "Quality inspection must be completed before a receipt can be issued"
A farmer can confirm / dispute only their own receipt	403
Only an <i>issued</i> receipt can be confirmed or disputed (no double confirmation)	409 "Receipt is already confirmed"
Dispute requires an explanation note	400 "Please explain what is wrong with the receipt"
Collection cannot be marked completed before the inspection is approved	400

4. Suggested live demo

1. Farmer logs in → **Schedule delivery**. Try tomorrow-minus-one date and a negative quantity to show the rules; then a valid booking (US1).
2. Officer opens **Delivery appointments**, confirms the appointment, then **Register collection** linking it (US2). Show that a walk-in works without an appointment.
3. **Weigh produce**: try container $\geq$ gross → error; then valid values and show the net weight appear (US3).
4. Inspector → **Pending inspections** → inspect: choose grade, enter accepted + rejected that do not equal net weight → refused; then correct values with a rejection reason and photos (US4, US5, US6, US9).
5. Show the success message with the new **batch number** and **receipt number**; open inventory to see the batch (US7).
6. Farmer opens **My collections** → receipt → **Confirm** (or dispute with a note) (US8).

5. Likely viva questions and model answers

Q1. How do you make sure the net weight is correct?

It is a generated column in PostgreSQL: gross minus container. Users only enter gross and container, and a CHECK constraint plus API validation ensure gross is greater than container.

Q2. What prevents an inspector from accepting more than was delivered?

`finalize_inspection()` requires accepted + rejected to equal the recorded net weight; otherwise it raises `INVALID_QTY` (HTTP 400). The inspection form enforces the same rule.

Q3. Why is inspection implemented as a database function?

It must create the inspection, rejections, batch, inventory ledger entry, receipt and status updates atomically. A single transaction guarantees there are never partial records.

Q4. What happens if two inspectors inspect the same collection?

The second call fails with `INVALID_STATE` "an inspection already exists" (409); the receipt table also has a `UNIQUE` `collection_id`.

Q5. How is the batch code created and what is it used for?

A database sequence function creates the batch number; the QR value is `HH-<batch_no>`. It links stock to the farmer and collection for traceability and drives expiry tracking.

Q6. How is the expiry date decided?

Each crop category has `shelf_life_days` (default 7; longer for rice, spices, tea). Expiry = today + shelf life when the batch is created.

Q7. What is the difference between a scheduled delivery and a walk-in?

A scheduled delivery references an appointment; a walk-in does not. Both create the same collection record afterwards.

Q8. How do you prevent overbooking?

The appointment API counts bookings for the centre and date against the daily capacity setting and returns 409 when full; it also blocks duplicate bookings by the same farmer.

Q9. Why do you use status transition tables?

To forbid impossible jumps (e.g. accepting before weighing). Every status change is checked against allowed next states and returns 409 otherwise.

Q10. Can the officer edit weights later?

Only while the collection has not been inspected. After that the API returns 409, so stored quantities cannot be changed behind an approved inspection.

Q11. What does the farmer get after inspection?

A numbered receipt (accepted / rejected kg, grade, batch number), an in-app notification (and email if SMTP is on), and the right to confirm or dispute it.

Q12. How are rejection reasons recorded?

One row per reason in `collection_rejections` with quantity and notes; at least one reason is mandatory when anything is rejected, and their quantities cannot exceed the rejected total.

Q13. What are the roles for each action?

Farmer books and confirms receipts; officer registers, weighs and issues receipts; inspector grades; the API uses `requireRole` so any other role gets 403.

6. Know your limitations

- There is no integration with a physical weighing scale – weights are entered manually (the Sprint 0 document allows a software simulator).

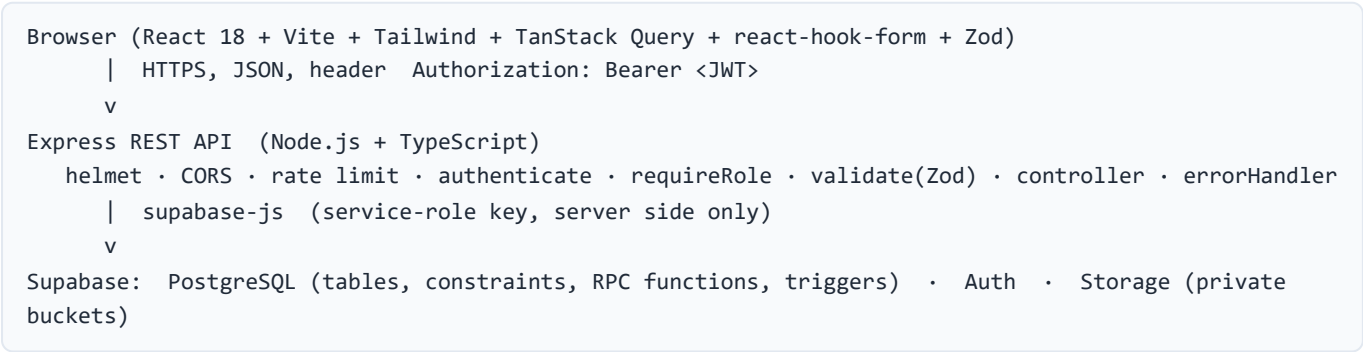
- The batch QR is stored as text (`HH-<batch_no>`); a scannable image/camera scan is not built.
- Evidence images are limited by the 10 MB request size of the API; very large photos should be compressed first.
- Automated tests do not yet cover the inspection function end to end; it was checked manually (the SQL test scripts in `supabase/tests/` cover stock reservation, wastage and payments).

7. Where to find it in the code

Concern	File
Appointments	<code>backend/src/controllers/appointmentController.ts</code>
Collections, weighing, receipts	<code>backend/src/controllers/collectionController.ts</code>
Inspections, images	<code>backend/src/controllers/inspectionController.ts</code>
Atomic inspection + batch + receipt	<code>supabase/migrations/025_receipts_inspection_payments.sql</code>
Net weight generated column	<code>supabase/migrations/001_initial_schema.sql</code> (produce_collections)
Error mapping	<code>backend/src/utis/rpcError.ts</code>
Frontend	<code>frontend/src/pages/farmer/*</code> , <code>pages/officer/*</code> , <code>pages/inspector/*</code> , <code>components/receipts/CollectionReceiptCard.tsx</code>

8. Appendix – system foundations (same in every guide)

8.1 Architecture in one picture



Layered design: presentation (React pages) → API/business layer (Express controllers and services) → data layer (Supabase/PostgreSQL). Business rules that must never be bypassed (stock, weights, payments) are enforced in the **database** through PostgreSQL functions, so even a buggy client cannot break them.

8.2 Authentication vs authorisation

Concept	What it answers	How HarvestHub does it
Authentication	"Who are you?"	Email + password are checked by Supabase Auth . On success the API returns a JWT access token and a refresh token. The frontend stores them and the Axios interceptor adds <code>Authorization: Bearer <token></code> to every request; on a 401 it tries the refresh token once, otherwise logs the user out.
Authorisation (RBAC)	"What may you do?"	<code>authenticate</code> middleware verifies the token with Supabase, loads the user's <code>profiles</code> row (role, account_status) and rejects suspended / inactive / pending accounts. <code>requireRole(...)</code> then allows only listed roles (administrator is a superuser). Wrong role → 403 .
Ownership checks	"Is this YOUR record?"	Role alone is not enough: a farmer must not read another farmer's data. Controllers call helpers such as <code>assertFarmerAccess</code> / <code>assertBuyerAccess</code> (services/access.ts) → 403 if the record belongs to someone else.
Row Level Security (RLS)	DB-level safety net	Policies are enabled on tables in Supabase. The API itself uses the service-role key, so the API-layer checks above are the primary control ; RLS protects direct database access.

The nine roles are: **farmer, collection_centre_officer, quality_inspector, inventory_manager, buyer, finance_officer, transport_coordinator, manager, administrator**. Only **farmer** and **buyer** can self-register; staff accounts are created by an administrator.

8.3 Three layers of validation

Layer	Tool	Purpose	Failure looks like
1. Frontend	Zod schema + react-hook-form	Instant feedback, fewer bad requests. Never trusted on its own.	Red message under the field
2. API	Zod schema via <code>validate()</code> middleware + controller rules	The real gate: format, required fields, ownership, state rules.	HTTP 400 with <code>{success:false, message, errors: [{field,message}]}</code>
3. Database	NOT NULL, UNIQUE, CHECK, enums, foreign keys, PL/pgSQL functions	Guarantees integrity even if layers 1–2 are bypassed.	Exception mapped to 404 / 400 / 409

Say this in the evaluation: "We validate on the client for usability, on the server for security, and in the database for integrity. The client is never trusted."

8.4 HTTP status codes used

Code	Meaning	Example in this system
200 / 201	OK / Created	Profile loaded; account or farmer created
400	Bad request – invalid input	Bad NIC format, weak password, quantity ≤ 0
401	Not authenticated	Missing / expired token, wrong password

Code	Meaning	Example in this system
403	Authenticated but not allowed	Wrong role, or a farmer opening another farmer's record, pending farmer logging in
404	Not found	Unknown farmer / order id
409	Conflict with current state	Duplicate NIC, duplicate booking, invalid status transition, insufficient stock
429	Too many requests	Rate limiter (e.g. crop advisor: 30 requests / 15 min)
500	Server error	Unexpected failure (logged by winston)

8.5 Cross-cutting security measures

- **helmet** security headers; **CORS** restricted to the configured frontend origin in production.
- **Rate limiting** with express-rate-limit (general API and a stricter limit on `/api/auth`).
- **Safe search**: user search text is stripped of characters that could alter a PostgREST filter (`( , ( ) % * \ )`) and limited to 60 characters.
- **Mass-assignment protection**: update endpoints copy only a whitelist of columns (a farmer cannot set their own verification status).
- **Generic login error** ("Invalid email or password") so attackers cannot discover which emails exist.
- **Secrets** only in `.env` (service-role key never reaches the browser); private storage buckets with short-lived signed URLs for documents.
- **Notifications and audit**: important actions create in-app notification rows (and emails when SMTP is configured); auction awards and rejections are written to the audit log.